

Android Studio 单元测试实践报告

一、实验目的

掌握在 Android Studio 中创建业务类并编写单元测试的方法。
学会使用 Android Studio 自动生成单元测试用例，验证代码逻辑的正确性。
理解单元测试的基本流程，包括用例编写、运行与结果分析。

二、实验环境

开发工具：Android Studio Hedgehog
开发语言：Kotlin
测试框架：JUnit 4
项目名称：UnitTestDemo

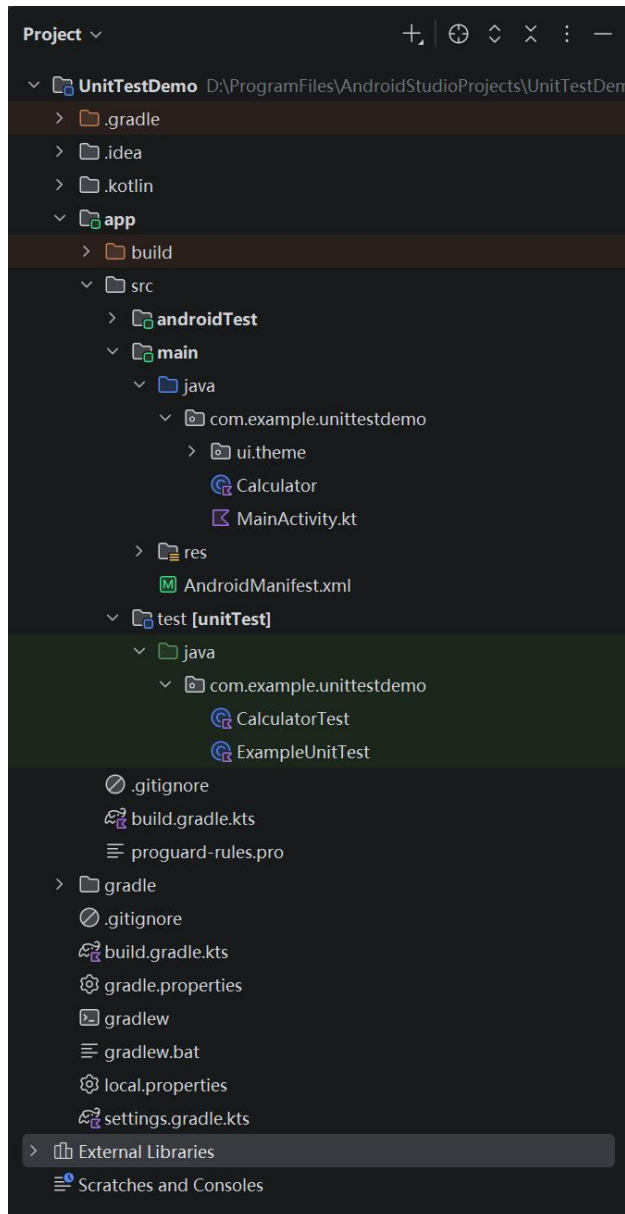
三、实验步骤与关键截图

步骤 1：创建 Android 项目

打开 Android Studio，新建 Empty Activity 项目，项目名称为 UnitTestDemo。
选择 Kotlin 语言，Minimum SDK 为 API 24，设置纯英文路径，避免空格和中文。

等待 Gradle 同步完成，项目构建成功。

截图 1：项目创建与目录结构



步骤 2：编写业务逻辑代码

在 `app/src/main/java/com/example/unittestdemo` 目录下创建 `Calculator.kt` 类。实现加减乘除 4 个基础运算方法，并对除法添加除数为 0 的异常处理。核心代码如下：

kotlin

```
class Calculator {  
    // 加法  
    fun add(a: Int, b: Int): Int {  
        return a + b  
    }  
  
    // 减法  
    fun subtract(a: Int, b: Int): Int {  
        return a - b  
    }  
}
```

```

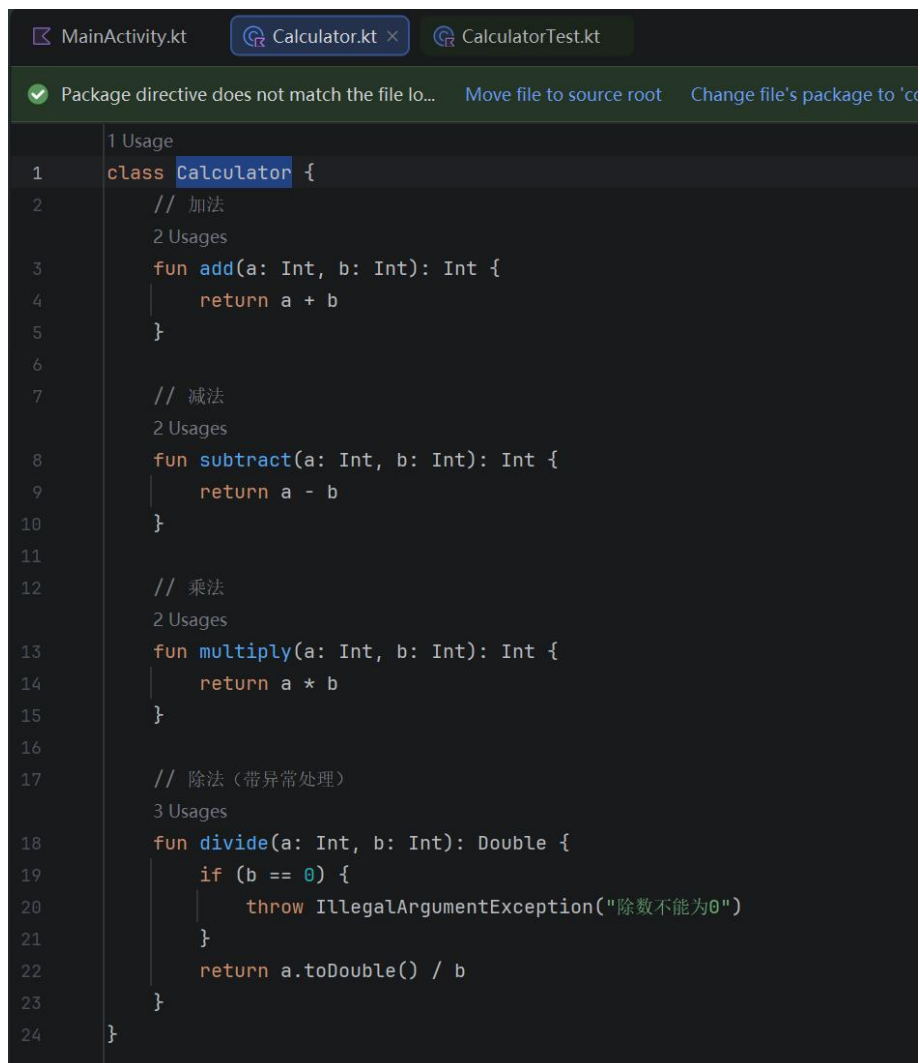
    }

    // 乘法
    fun multiply(a: Int, b: Int): Int {
        return a * b
    }

    // 除法（带异常处理）
    fun divide(a: Int, b: Int): Double {
        if (b == 0) {
            throw IllegalArgumentException("除数不能为 0")
        }
        return a.toDouble() / b
    }
}

```

截图 2: Calculator 业务代码



步骤 3: 自动生成单元测试用例

选中 Calculator 类，右键点击 Generate → Test。

选择测试框架为 JUnit 4，目标包为 app/src/test/java/com/example/unittestdemo。

勾选所有 4 个方法，生成测试类 CalculatorTest.kt。

完善测试用例，添加边界条件和异常测试。

核心测试代码如下：

```
kotlin
import org.junit.Test import org.junit.Assert.*
class CalculatorTest {
    private val calculator = Calculator()

    @Test
    fun add() {
        assertEquals(5, calculator.add(2, 3))
        assertEquals(-1, calculator.add(-5, 4))
    }

    @Test
    fun subtract() {
        assertEquals(1, calculator.subtract(3, 2))
        assertEquals(-7, calculator.subtract(-5, 2))
    }

    @Test
    fun multiply() {
        assertEquals(6, calculator.multiply(2, 3))
        assertEquals(0, calculator.multiply(5, 0))
    }

    @Test
    fun divide() {
        assertEquals(2.0, calculator.divide(4, 2), 0.001)
        assertEquals(2.5, calculator.divide(5, 2), 0.001)
    }

    @Test(expected = IllegalArgumentException::class)
    fun divideByZeroThrowsException() {
        calculator.divide(5, 0)
    }
}
```

截图 3: CalculatorTest 测试代码

```
MainActivity.kt Calculator.kt CalculatorTest.kt x
Package directive does not match the file lo... Move file to source root Change file's package to 'com.example.unittestdemo'

1 import org.junit.Test
2 import org.junit.Assert.*
3
4 class CalculatorTest {
5     9 Usages
6     private val calculator = Calculator()
7
8     @Test
9     fun add() {
10         assertEquals( expected = 5, actual = calculator.add( a = 2, b = 3))
11         assertEquals( expected = -1, actual = calculator.add( a = -5, b = 4))
12     }
13
14     @Test
15     fun subtract() {
16         assertEquals( expected = 1, actual = calculator.subtract( a = 3, b = 2))
17         assertEquals( expected = -7, actual = calculator.subtract( a = -5, b = 2))
18     }
19
20     @Test
21     fun multiply() {
22         assertEquals( expected = 6, actual = calculator.multiply( a = 2, b = 3))
23         assertEquals( expected = 0, actual = calculator.multiply( a = 5, b = 0))
24     }
25
26     @Test
27     fun divide() {
28         assertEquals( expected = 2.0, actual = calculator.divide( a = 4, b = 2), delta = 0.001)
29         assertEquals( expected = 2.5, actual = calculator.divide( a = 5, b = 2), delta = 0.001)
30     }
31
32     @Test(expected = IllegalArgumentException::class)
33     fun divideByZeroThrowsException() {
34         calculator.divide( a = 5, b = 0)
35     }
36 }
```

步骤 4：运行单元测试并验证结果

右键点击 CalculatorTest.kt 文件，选择 Run 'CalculatorTest'。

查看底部 Run 窗口，等待测试执行完成。

验证所有测试用例是否通过，本次实验中 5 个测试用例全部通过，耗时 4ms。

截图 4：单元测试运行结果

```
Run CalculatorTest
Test Results 4ms
5 tests passed 5 tests total, 4ms
Executing tasks: [:app:testDebugUnitTest, --tests, CalculatorTest] in project D:\ProgramFiles\AndroidStudioProjects\UnitTestDemo

> Task :app:preBuild UP-TO-DATE
> Task :app:preDebugBuild UP-TO-DATE
> Task :app:generateDebugResources
> Task :app:packageDebugResources
> Task :app:processDebugNavigationResources
> Task :app:javaPreCompileDebug
> Task :app:parseDebugLocalResources
> Task :app:checkDebugAarMetadata
> Task :app:mapDebugSourceSetPaths
> Task :app:generateDebugFile
Download https://dl.google.com/dl/android/maven2/com/android/tools/build/aapt2/9.2.1-15009934/aapt2-9.2.1-15009934.jar, took 415 ms
> Task :app:compileDebugNavigationResources
```

四、实验结果分析

本次单元测试对 Calculator 类的 4 个方法进行了全面验证：

1. 加减乘除基础运算的正确性测试通过。
2. 除法除数为 0 的异常处理测试通过。

3. 边界条件（如乘法与 0 相乘、负数运算）测试通过。
4. 所有用例均无报错，说明业务逻辑代码编写正确，单元测试有效验证了代码的健壮性。

五、实验总结

通过本次实验，我掌握了 Android Studio 中单元测试的完整流程，理解了 JUnit 测试框架的使用方法，学会了自动生成测试用例并完善边界条件测试。单元测试能够帮助开发者快速发现代码中的逻辑错误，提高代码质量，是软件开发中不可或缺的环节。